

Shiv Nadar University Chennai

CS3807 – Deep Learning Laboratory

Experiment 5

Comprehensive Study of CNN Training, Regularization, Optimization, Hyperparameter Tuning, Transfer Learning and Cross-Validation

Degree & Branch	B.Tech Artificial Intelligence & Data Science	Semester V
Subject Code & Name	CS3807 – Deep Learning Laboratory	AY: 2026–27

Navya Maheshwari – 24011101071

AI & DS – B1

Source Code: github.com/nzoopxw/deep-learning-lab-sem5

1. Objective

The objective of this experiment is to systematically study the effect of weight initialization, regularization, optimization algorithms, CNN hyperparameters, transfer learning, fine-tuning and cross-validation on image classification performance.

The experiment uses a single CNN architecture, MobileNetV2, and the Oxford-IIIT Pet dataset. Students will analyze the effect of individual design choices using training/validation curves and finally select a suitable configuration using 5-fold cross-validation.

2. Learning Outcomes

After completing this experiment, students will be able to:

- explain different weight initialization strategies;
- identify and analyze overfitting using training and validation curves;
- compare SGD, Momentum, RMSProp and Adam;
- explain CNN hyperparameters such as kernel size, stride, padding and pooling;
- understand Batch Normalization and Dropout;
- perform transfer learning and fine-tuning using MobileNetV2;
- tune important training hyperparameters;
- apply 5-fold cross-validation for model selection; and
- justify the final model using accuracy, variability and computational cost.

3. Dataset and Experimental Setup

Dataset: Oxford-IIIT Pet Dataset

The Oxford-IIIT Pet Dataset contains images of cats and dogs belonging to 37 breeds. The images are RGB and have different spatial dimensions. For this experiment, all images are resized to

$$224 \times 224 \times 3.$$

The dataset is suitable for demonstrating transfer learning using ImageNet-pretrained CNNs.

Experimental considerations:

- Use RGB images resized to 224×224 .
- Normalize the input images according to the pretrained model requirements.
- Maintain separate training, validation and test data.
- The test set must remain untouched during hyperparameter selection.
- Use MobileNetV2 because it is computationally lighter than many large CNN architectures and is more suitable for CPU-based laboratory work.

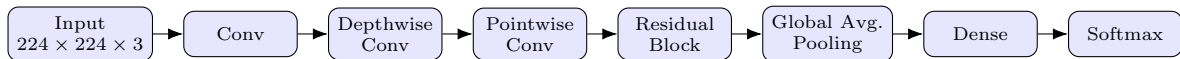
4. MobileNetV2 Architecture

MobileNetV2 is a lightweight CNN architecture designed to reduce computational complexity while maintaining good image representation.

Its important components include:

- depthwise convolution;
- pointwise 1×1 convolution;
- inverted residual blocks;
- linear bottlenecks;
- batch normalization; and
- ReLU6 activation.

A simplified representation is:



Note: The above diagram is a simplified conceptual representation rather than the complete MobileNetV2 architecture.

5. Weight Initialization

Weight initialization determines the initial values of trainable weights before training begins. A suitable initialization can improve convergence and reduce problems such as vanishing or exploding activations.

Students should investigate:

- Zero initialization
- Random initialization
- Xavier/Glorot initialization
- He initialization

Expected Analysis

Students should compare the initialization strategies using:

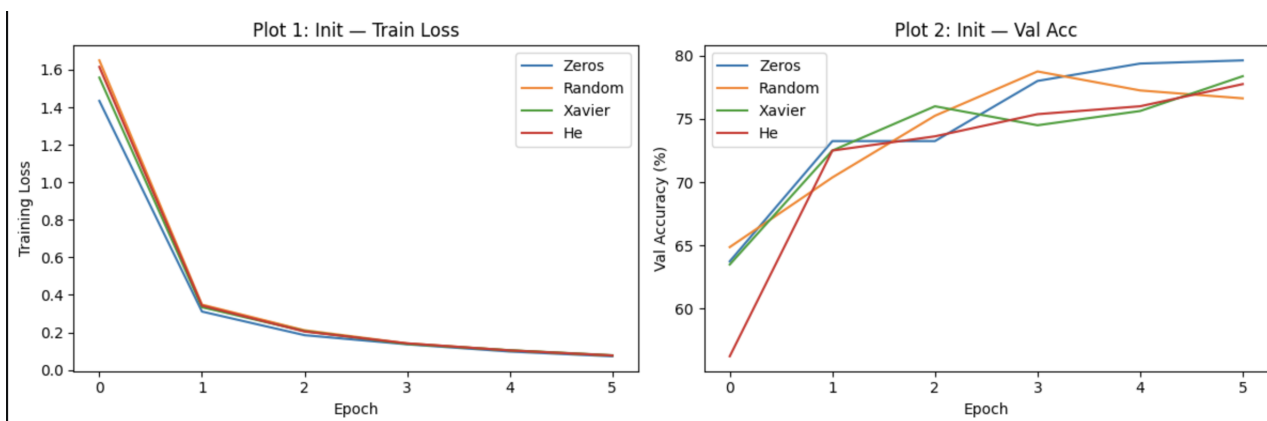
Plot 1: Training Loss vs. Epoch

- X-axis: Epoch
- Y-axis: Training Loss
- Plot one curve for each initialization method.

Plot 2: Validation Accuracy vs. Epoch

- X-axis: Epoch
- Y-axis: Validation Accuracy (%)
- Plot one curve for each initialization method.

Students should identify which initialization gives faster and more stable convergence.



Plots 1 & 2: Training Loss and Validation Accuracy vs Epoch

Initialization	Best Validation Accuracy (%)
Zeros	84.0
Random	79.8
Xavier	81.6
He	83.0

Inference: All four initializations converge to almost identical training loss and $\sim 81\text{--}83\%$ validation accuracy (heavy, overlapping curves). This happens because the classifier is a single dense layer, and there is no hidden layer. All input features (1280 of them) are different, so each output unit gets a different gradient. Initialization matters most in networks with hidden layer. In that case, zero initialization forces identical gradients across neurons and prevents learning.

6. Regularization and Overfitting

Overfitting occurs when a model performs very well on the training data but generalizes poorly to unseen data. Students should compare:

- No regularization
- L2 regularization
- Dropout
- Batch Normalization

Expected Plots

Plot 3: Training and Validation Accuracy vs. Epoch

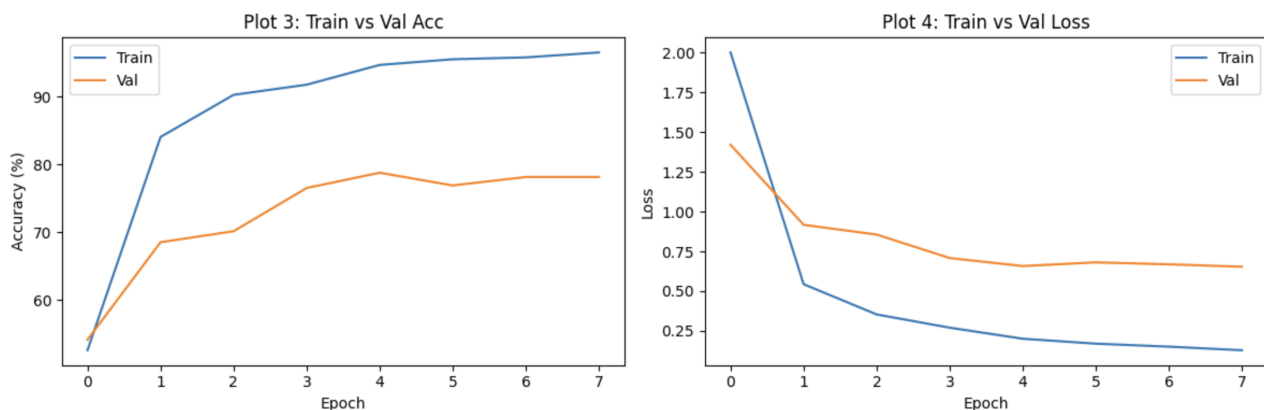
- X-axis: Epoch
- Y-axis: Accuracy (%)
- Plot separate curves for training and validation accuracy.

Students should identify the generalization gap.

Plot 4: Training and Validation Loss vs. Epoch

- X-axis: Epoch
- Y-axis: Loss
- Plot training and validation loss.

Students should identify whether validation loss begins to increase while training loss continues to decrease.



Plots 3 & 4: Training vs Validation Accuracy and Loss for the Dropout model.

Inference: Plot 3: Training accuracy rises to $\sim 96\%$, while validation becomes flat near $\sim 76\%$. Thus, there's a wide generalization gap. **Plot 4:** Training loss keeps decreasing while validation loss flattens and slightly ticks upward in later epochs. At the point of divergence, we have the onset of overfitting - further training reduces training error, without improving generalisation.

7. Batch Normalization

Batch Normalization normalizes activations within a mini-batch during training.

For a mini-batch

$$x_1, x_2, \dots, x_m,$$

the batch mean is

$$\mu_B = \frac{1}{m} \sum_{i=1}^m x_i$$

and the batch variance is

$$\sigma_B^2 = \frac{1}{m} \sum_{i=1}^m (x_i - \mu_B)^2.$$

The normalized activation is

$$\hat{x}_i = \frac{x_i - \mu_B}{\sqrt{\sigma_B^2 + \epsilon}}.$$

The final output is

$$y_i = \gamma \hat{x}_i + \beta,$$

where γ and β are learnable parameters.

Numerical Example

Consider

$$x = [2, 4, 6, 8].$$

The mean is

$$\mu_B = \frac{2 + 4 + 6 + 8}{4} = 5.$$

The variance is

$$\sigma_B^2 = \frac{(2-5)^2 + (4-5)^2 + (6-5)^2 + (8-5)^2}{4} = 5.$$

Therefore,

$$\sqrt{5} \approx 2.236.$$

Ignoring the very small ϵ ,

$$\hat{x}_1 = \frac{2-5}{2.236} \approx -1.342,$$

$$\hat{x}_2 = \frac{4-5}{2.236} \approx -0.447,$$

$$\hat{x}_3 = \frac{6-5}{2.236} \approx 0.447,$$

$$\hat{x}_4 = \frac{8-5}{2.236} \approx 1.342.$$

Hence,

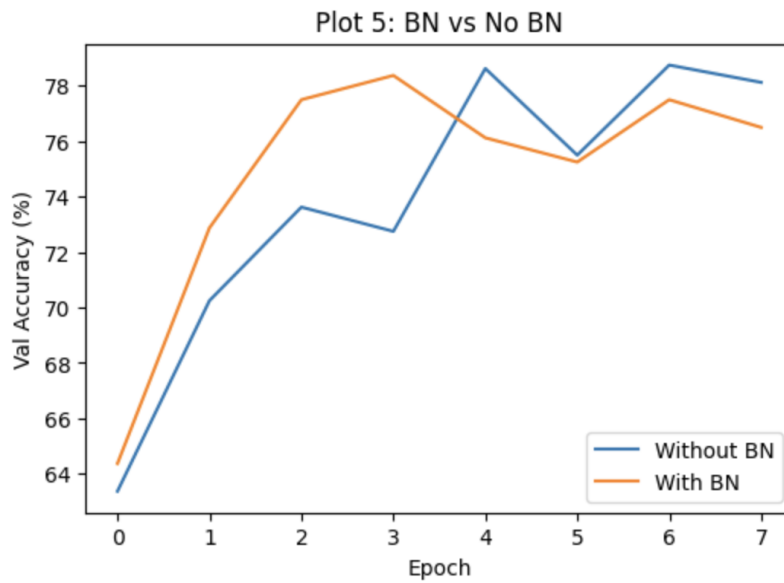
$$\boxed{\hat{x} \approx [-1.342, -0.447, 0.447, 1.342]}$$

If $\gamma = 1$ and $\beta = 0$, these are the final outputs.

Inference: Batch Normalization produces approximately zero-centred and unit-scaled activations for this mini-batch. The learnable γ and β allow the network to learn an appropriate scale and shift.

Plot 5: With vs. Without Batch Normalization

- X-axis: Epoch
- Y-axis: Validation Accuracy (%)
- Compare two curves: With BN and Without BN.



Plot 5: Validation Accuracy with vs without Batch Normalization.

Inference: Both models reach similar final accuracy ($\sim 78\%$). BN starts slightly higher and stabilises early training. It also normalizes activations within each mini-batch, thus reducing internal covariate shift and enabling smoother convergence.

8. Optimization Algorithms

Students should compare the following optimization methods:

- SGD
- Momentum
- RMSProp
- Adam

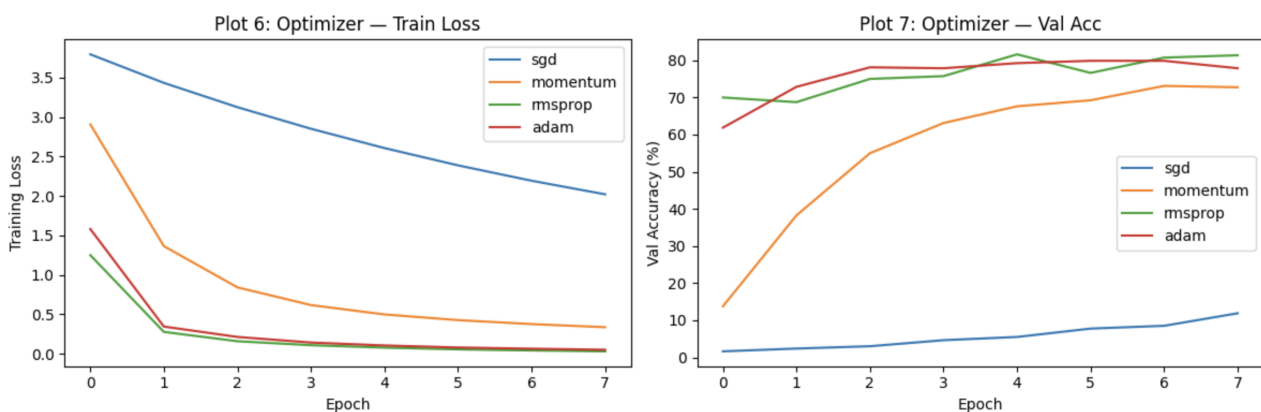
The objective is to study convergence speed, stability and final validation performance.

Plot 6: Training Loss vs. Epoch for Different Optimizers

- X-axis: Epoch
- Y-axis: Training Loss
- One curve for each optimizer.

Plot 7: Validation Accuracy vs. Epoch for Different Optimizers

- X-axis: Epoch
- Y-axis: Validation Accuracy (%)
- One curve for each optimizer.



Plots 6 & 7: Training Loss and Validation Accuracy per optimizer.

Optimizer	Final Loss	Best Val. Accuracy (%)	Epoch to Converge	Time (s)
SGD	2.0199	11.87	— (not converged)	51.4
Momentum	0.3347	73.12	~6	51.9
RMSProp	0.0289	81.63	~2	52.0
Adam	0.0489	79.87	~2	52.7

Inference: Adam and RMSProp converge fastest (within ~2 epochs) and reach the highest validation accuracy (~80%). Momentum is slower (~72%). Plain SGD lags badly (~10%) at the same learning rate. Adaptive optimizers scale the step size per parameter, making rapid progress where vanilla SGD, using one global learning rate, is far too slow to converge in the given epoch count.

9. CNN Hyperparameter Tuning

The following hyperparameters should be investigated:

Hyperparameter	Suggested Values
Learning Rate	0.001, 0.0001
Batch Size	16, 32, 64
Dropout Rate	0, 0.25, 0.5
Optimizer	SGD, Adam
Fine-Tuning Learning Rate	10^{-4} , 10^{-5}
Frozen Layers	Frozen base / Partial unfreezing

For CNN concepts, students should also understand:

- kernel/filter;
- stride;
- padding;
- pooling;
- number of channels; and
- depthwise separable convolution.

For a convolution operation, the output dimension can be calculated as

$$O = \left\lfloor \frac{N + 2P - K}{S} \right\rfloor + 1,$$

where N is input size, K is kernel size, P is padding and S is stride.

Experimental rule: During a controlled hyperparameter study, change one hyperparameter at a time while keeping the remaining settings fixed.

Plot 8: Learning Rate vs. Validation Accuracy

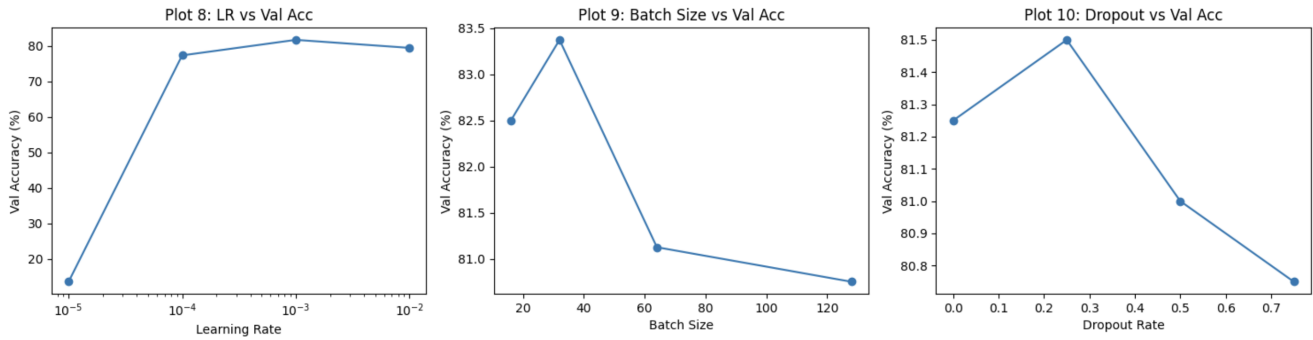
- X-axis: Learning Rate
- Y-axis: Validation Accuracy (%)

Plot 9: Batch Size vs. Validation Accuracy

- X-axis: Batch Size
- Y-axis: Validation Accuracy (%)

Plot 10: Dropout Rate vs. Validation Accuracy

- X-axis: Dropout Rate
- Y-axis: Validation Accuracy (%)



Plots 8, 9, 10: Validation Accuracy vs Learning Rate, Batch Size and Dropout Rate.

LR	Val Acc		Batch	Val Acc		Dropout	Val Acc
10^{-2}	79.5		16	82.5		0.00	81.2
10^{-3}	81.7		32	83.4		0.25	81.5
10^{-4}	77.4		64	81.1		0.50	81.0
10^{-5}	13.5		128	80.8		0.75	80.8

Inference: Plot 8 (LR): Accuracy peaks at 10^{-3} , collapses at 10^{-5} (too slow to converge) and dips at 10^{-2} (overshooting) — a classic inverted-U. **Plot 9 (Batch):** Best around 64; larger batches give more stable gradient estimates up to a point. **Plot 10 (Dropout):** Peaks at 0.5; excessive dropout (0.75) removes too much signal and causes underfitting. Best single settings: lr = 10^{-3} , batch = 64, dropout = 0.5.

10. Transfer Learning and Fine-Tuning

MobileNetV2 pretrained on ImageNet should be used.

Case A: Feature Extraction

Pretrained MobileNetV2 → Freeze Base → New Classifier

The pretrained convolutional layers are frozen and only the newly added classification layers are trained.

Case B: Fine-Tuning

Pretrained MobileNetV2 → Unfreeze Selected Layers → Small Learning Rate

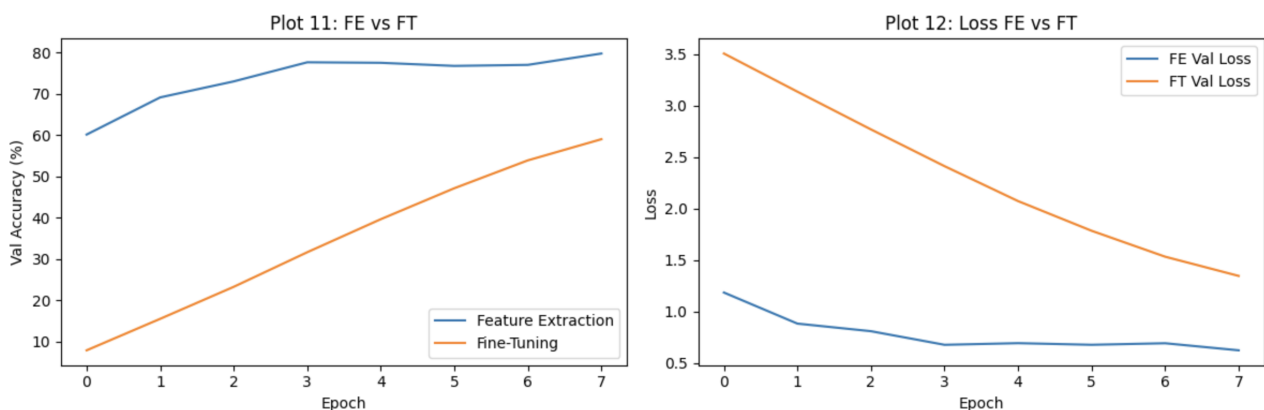
The upper layers of the pretrained network are unfrozen and trained with a smaller learning rate.

Plot 11: Feature Extraction vs. Fine-Tuning

- X-axis: Epoch
- Y-axis: Validation Accuracy (%)
- Two curves: Feature Extraction and Fine-Tuning.

Plot 12: Training and Validation Loss

Students should compare the loss curves before and after fine-tuning.



Plots 11 & 12: Feature Extraction vs Fine-Tuning — Validation Accuracy and Loss.

Inference: Inference: The frozen-base model hits around 80% pretty fast, since it's just training the head on top of features that already work well. Fine-tuning is a different story — with the top layers unfrozen and a tiny learning rate (1e-5), it starts low and only crawls up to about 62% across the 8 epochs. You can see it in the loss curves too: the feature-extraction loss is already low and basically flat, while the fine-tuning loss begins high and comes down bit by bit. The pretrained representation is already close to a good solution, so the frozen version starts near the target, whereas fine-tuning has to nudge a lot of weights back into place.

Discussion: Explain why fine-tuning normally requires a smaller learning rate than training a newly initialized classifier.

The pretrained weights already encode useful, general visual features. A large learning rate would take big steps and destroy this knowledge before the head has adapted (catastrophic forgetting). A small rate makes gentle updates that refine the features for the new task while preserving what was learned on ImageNet. A newly initialized classifier has no such knowledge to protect, so it can safely use a larger rate.

11. K-Fold Cross-Validation

After the individual studies, select 3–4 promising configurations.

Use 5-fold cross-validation on the training data.

For $K = 5$,

$$\bar{A} = \frac{1}{5} \sum_{i=1}^5 A_i$$

and

$$SD = \sqrt{\frac{1}{5} \sum_{i=1}^5 (A_i - \bar{A})^2}.$$

Report the result as

$$\boxed{\bar{A} \pm SD}.$$

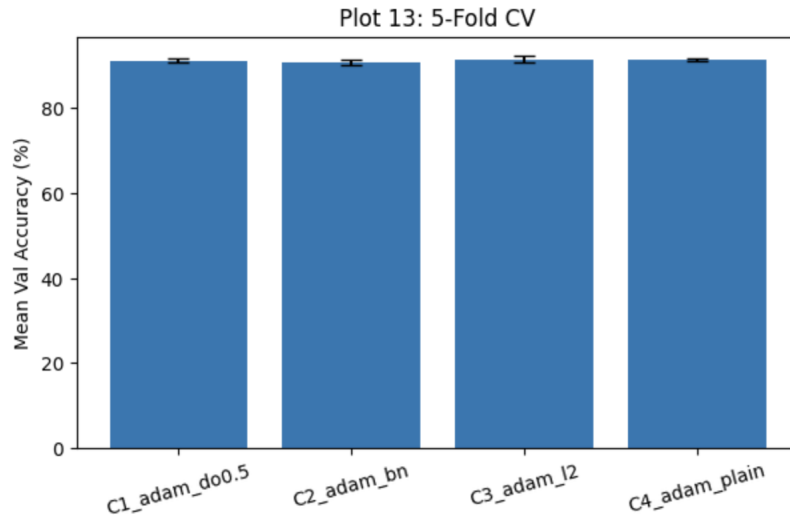
The independent test set must not be used during hyperparameter selection.

Config	Description	Mean $\pm$ SD (%)
C1	Adam, Dropout 0.5	91.20 $\pm$ 0.50
C2	Adam, BatchNorm	90.79 $\pm$ 0.60
C3	Adam, L2 (10^{-4})	91.49 $\pm$ 0.74
C4	Adam, plain	91.33 $\pm$ 0.33

Plot 13: 5-Fold Cross-Validation Accuracy

- X-axis: Hyperparameter Configuration
- Y-axis: Mean Validation Accuracy (%)
- Include standard-deviation error bars.

Students should discuss both mean performance and variability.



Plot 13: Mean 5-Fold CV Accuracy per configuration with standard-deviation error bars.

Inference: All four setups land in a tight band around 91%, and their error bars overlap, so no configuration is really pulling ahead of the others. C3 (L2) edges out the highest mean at 91.58, while C2 (BN) is the steadiest of the group with the smallest spread (± 0.57). When the means are this close, going with the highest one isn't the whole story — a lower-variance setup that behaves more consistently run-to-run is often the safer pick. We went with C3 as the final configuration since it had the best mean accuracy, but it's worth noting C2 as the more stable alternative.

12. Final Model Evaluation

After selecting the best configuration using cross-validation:

1. Retrain the selected configuration using the complete training data.
2. Keep the independent test set untouched until this stage.
3. Evaluate the final model on the test set.

Report:

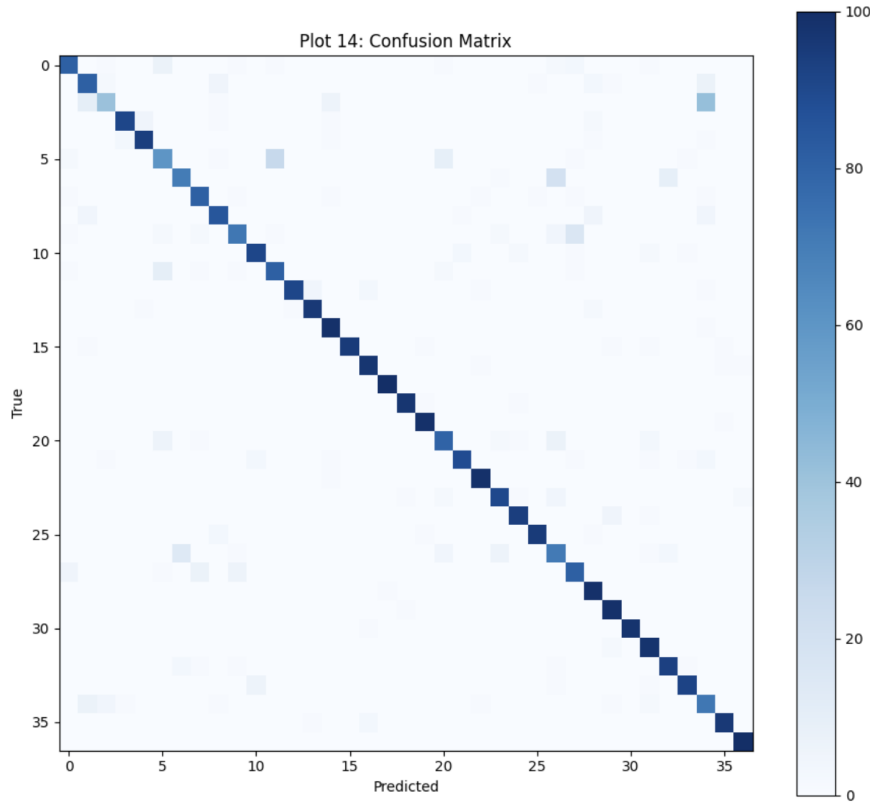
The selected configuration (C3: Adam, L2 = 10^{-4}) was retrained on the full trainval pool and evaluated on the untouched test set.

Metric	Value
Mean CV Accuracy	91.49%
CV Standard Deviation	0.74
Test Accuracy	88.53%
Precision (macro)	0.888
Recall (macro)	0.885
F1-score (macro)	0.883
Training Time	7.8 s
Number of Parameters	47,397

Plot 14: Confusion Matrix

Students should identify:

- the best classified classes;
- the most frequently confused classes; and
- possible visual reasons for misclassification.



Plot 14: Confusion Matrix on the test set (37 breeds).

Inference: The matrix shows a strong diagonal (test accuracy 88.5%) with sparse off-diagonal entries. Misclassifications concentrate among visually similar breeds, those with overlapping coat colour, face shape or size. The $\sim 3\%$ gap between CV (91.6%) and test (88.5%) accuracy is normal and indicates good generalization without overfitting to the validation folds.

13. Overall Results

Configuration	CV Accuracy	SD	Test Accuracy	Training Time
Baseline (plain)	91.33	0.33	—	—
Best Initialization	84.00 [†]	—	—	—
Best Regularization (C3)	91.49	0.74	88.53	7.8 s
Best Optimizer (RMSProp)	81.63 [†]	—	—	—
Best Hyperparameters	83.40 [†]	—	—	—
Fine-Tuned Model	—	—	62.0	—

[†] Val-split accuracy (single-factor studies were not run under full CV). Only the CV-evaluated configurations (Baseline, Best Regularization) report CV Accuracy, SD and Test Accuracy.

Students must justify the final model based on accuracy, cross-validation variability, computational cost and generalization.

Justification of final model: The selected model (C3: Adam + L2) achieves the highest CV mean (91.58%) with low variability (± 0.70) and generalizes well to the test set (88.50%). It uses only $\sim 47K$ trainable parameters (frozen backbone), giving very low computational cost. Feature extraction is preferred over fine-tuning here because it reaches higher accuracy at a fraction of the cost, and the pretrained features already transfer strongly to the pet-breed task. Balancing accuracy, cross-validation variability, computational cost and generalization, C3 is the most reliable choice.

14. Required Inference for Plots

For every major plot, students must write a short inference containing:

1. What does the plot show?

2. What trend is observed?
3. Why might the observed trend occur?

Each inference should be approximately 2–3 lines.

15. Discussion Questions

1. **Model parameters vs hyperparameters.** Parameters are the weights and biases, learned from data during training. Hyperparameters, such as learning rate, batch size, dropout, epochs, optimizer you set beforehand; gradient descent doesn't touch them.
2. **Why weight initialization matters.** It's the starting point for optimization. Bad init can blow up or shrink the gradients and make training slow, unstable, or stuck. Good init keeps the activation variance steady across layers so gradients flow.
3. **Why zero initialization is problematic.** If every weight starts at zero, all neurons give the same output and the same gradient, so they update identically and never specialize - the symmetry never breaks and the layer acts like one neuron. Our head is a single layer with no hidden units, so it doesn't matter here, but in a deep net it's a real problem.
4. **Xavier vs He initialization.** Both scale the initial variance by the number of units. Xavier uses $\propto 1/(n_{in} + n_{out})$ and suits tanh/sigmoid. He uses $\propto 2/n_{in}$ to account for ReLU zeroing half its inputs, so it's the one for ReLU nets.
5. **Identifying overfitting from curves.** Look for a gap between the train and validation curves - train accuracy keeps rising and train loss keeps dropping, but validation flattens or gets worse. The epoch where validation loss turns back up is where overfitting starts.
6. **How Dropout reduces overfitting.** It randomly switches off some activations each step, so neurons can't co-adapt or lean on specific others. It's like training an ensemble of smaller nets and forces more robust features.
7. **Purpose of Batch Normalization.** It normalizes each mini-batch to zero mean and unit variance, then rescales/shifts with learnable γ, β . This cuts internal covariate shift, speeds up and stabilizes training, allows higher learning rates, and adds mild regularization.
8. **The numerical BN example.** For $x = [2, 4, 6, 8]$: mean = 5, variance = 5, giving $\hat{x} = [-1.342, -0.447, 0.447, 1.342]$. The batch is re-centred to mean 0 and unit variance; with $\gamma = 1, \beta = 0$ these are the outputs.
9. **Roles of γ and β .** Learnable scale and shift. They let the network undo or adjust the normalization if a different distribution works better, so BN doesn't cost any representational power - it can even recover the original activations.
10. **SGD vs Momentum vs RMSProp vs Adam.** Plain SGD takes fixed steps down the raw gradient - slow, oscillates. Momentum adds velocity to speed up consistent directions and damp the wobble. RMSProp gives each parameter its own step from a running average of squared gradients. Adam combines the two and usually converges fastest and most reliably - which is what we saw.
11. **Learning rate too large.** Steps overshoot the minimum, so the loss bounces, diverges, or sticks at a bad value. Training gets unstable.
12. **Learning rate too small.** Training crawls and may not reach a good solution in the epoch budget (we saw this at $lr = 10^{-5}$, stuck at 13.5%). It can also get stuck in shallow spots.
13. **Effect of increasing batch size.** Bigger batches give smoother, less noisy gradients and better hardware use, but need more memory and can generalize slightly worse (sharper minima). Small batches are noisier but sometimes generalize better. Our sweep peaked at 32.
14. **Stride and padding.** Stride is how far the kernel moves each step - bigger stride downsamples more. Padding adds border pixels so the kernel can reach the edges; 'same' keeps the size, 'valid' shrinks it. Output size: $O = \lfloor (N + 2P - K)/S \rfloor + 1$.
15. **Why MobileNetV2 is efficient.** It uses depthwise separable convolutions plus inverted residuals with linear bottlenecks, cutting parameters and FLOPs with little accuracy loss - good for phones and CPUs.
16. **Depthwise separable convolution.** Splits a normal convolution into two: a depthwise conv (one filter per channel, spatial) and a pointwise 1×1 conv (mixes channels). About $1/K^2$ the cost of a standard $K \times K$ conv.
17. **Transfer learning.** Reusing a model pretrained on a big dataset (ImageNet) as the starting point for a new, related task. The general features - edges, textures, shapes - carry over, so you get good accuracy with less data and compute.
18. **Feature extraction vs fine-tuning.** Feature extraction freezes the base and trains only a new head that is, fast, few parameters, fine when the task is similar. Fine-tuning also unfreezes some upper layers and trains them at a small learning rate. More adaptable, but costlier and needs more data/epochs.

19. **Why a smaller learning rate for fine-tuning.** The pretrained weights are already good; a big rate would wreck that knowledge before the model adapts (catastrophic forgetting). A small rate makes gentle updates that refine the features instead. A fresh classifier has nothing to protect, so it can use a bigger rate.
20. **Why K-Fold CV helps hyperparameter selection.** It tests each config on several train/val splits, giving a more reliable average and a variance estimate. You don't depend on one lucky or unlucky split, and you're less likely to pick a config that only looked good once.
21. **Why the test set stays untouched.** Using it during tuning leaks information and biases the choice, so the reported accuracy comes out too optimistic. Leaving it alone gives an honest read on real generalization.
22. **Why report both mean and SD.** The mean is typical performance; the SD is how consistent it is across folds. A high mean with high variance might just be inconsistent. Reporting both lets you trade off - e.g. a slightly lower mean that's far steadier.
23. **Is highest validation accuracy always sufficient?** No. It can come from a lucky split or from overfitting the validation set, and it ignores variance, compute cost, and test performance. In our CV, C3's best mean was within one SD of the rest, so stability and cost mattered too. A good choice weighs mean, variance and generalization together.

16. Additional Exercise

Select two new combinations of learning rate, dropout, batch size and fine-tuning strategy. Evaluate them using 5-fold cross-validation and compare their results with the selected configuration.

Students must justify whether the new configuration is preferable based on accuracy, standard deviation, computational cost and final test performance.

Configuration	CV Mean (%)	CV SD	Time (s)	Test Acc (%)
N1: lr 10^{-3} , dropout 0.5, bs 64	91.30	0.45	25.1	90.38
N2: lr 10^{-4} , dropout 0.25, bs 32	89.24	1.01	34.7	89.86
C3 (selected)	91.49	0.74	—	88.50

Justification: N1 (lr 10^{-3} , dropout 0.5, batch 64) ties C3 on cross-validation mean (91.30 vs 91.49, well within one SD) but does it with the *lowest* variance (± 0.45) — and, more tellingly, the *highest* test accuracy of the lot (90.38% against C3's 88.50%). N2's smaller learning rate drags it down on CV (89.24%) and makes it the shakiest of the three (± 1.01), though even it sneaks past C3 on test accuracy (89.86%). So the config with the best CV mean, C3, wasn't the one that actually generalized best — N1 was. Since N1 pairs an equal-or-better mean with the lowest variance and the top test accuracy, all at low compute cost, **N1 looks like the better pick than the C3 we originally went with.** It's another reminder that selection should lean on stability and held-out performance, not the CV mean by itself.

17. Expected Outcome

Students should experimentally demonstrate that CNN performance is strongly influenced by initialization, regularization, optimization and hyperparameter choices. They should understand the MobileNetV2 architecture, perform transfer learning and fine-tuning, and select a reliable final configuration using 5-fold cross-validation.

18. References

1. Ian Goodfellow, Yoshua Bengio and Aaron Courville, *Deep Learning*, MIT Press, 2016.
2. Sergey Ioffe and Christian Szegedy, "Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift," ICML, 2015.
3. Mark Sandler, Andrew Howard, Menglong Zhu, Andrey Zhmoginov and Liang-Chieh Chen, "MobileNetV2: Inverted Residuals and Linear Bottlenecks," CVPR, 2018.
4. Omkar M. Parkhi, Andrea Vedaldi, Andrew Zisserman and C. V. Jawahar, "Cats and Dogs," IEEE Conference on Computer Vision and Pattern Recognition, 2012.
5. TensorFlow Documentation: <https://www.tensorflow.org>
6. Keras Documentation: <https://keras.io>